Field Hours

CSCE 706 Structured Programming in C

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Compile, link, and a documented main

You start from a text file. The preprocessor runs first and pastes each header you name. The compiler then checks types and writes an object file. The linker joins that file to the C library and produces one program. Those are three tools, and a 12-line file that prints a single integer still goes through all of them. Stop after the compiler and you have no command you can run.

Inside main, statements run from the first line downward. The declaration int n = 4; creates an int and stores 4 in it. The next statement may read n. A comment does not run. Above main, you write two lines that name the input and the one result. That note is documentation. The compiler skips it, and you should not have to guess what the stored 4 is for.

Three tools, then a command.

Preprocessor pastes headers -> Compiler writes an object file -> Linker produces one program

Print the sum of 2 and 2

1. Inside main, declare int a = 2 and int b = 2.

2. Declare int s = a + b. The stored value is 4.

3. Pass s to printf with a format for one int. The program prints 4 and main returns 0.

Problem 1. You produce an object file and stop. Why is there still no program to run?

Answer. The linker has not joined that object file to the C library. You have the compiler's output, not a finished command.

Problem 2. The comment above main says the program prints 4, but the code adds a = 2 and b = 3. What does the machine print?

Answer. It prints 5. Comments are documentation and do not run. The stored values are 2 and 3, and their sum is 5.

Lecture 2 · 90 minutes

Integer division and operator order

An int on a common machine occupies 4 bytes, which is 32 bits, and one of those bits is the sign. You store a whole number, not a fraction. The expression 7 / 2 uses integer division and yields 3, because the remainder is dropped. The expression 7 % 2 yields 1, the remainder. If you needed the half that disappeared, a wider int would not restore it. You would use a double.

You do not choose precedence while you type. Multiplication happens before addition, so 2 + 3 * 4 is 14, not 20. Parentheses force the other grouping: (2 + 3) * 4 is 20. Assignment stores. The test == compares and does not store. If you write if (n = 4), you store 4, and the condition is true because 4 is not 0. That bug still compiles.

Same symbols, two groupings.

No parentheses

2 + 3 * 4

Multiply, then add

The result is 14

Parentheses first

(2 + 3) * 4

Add, then multiply

The result is 20

Where 7 sits on a 4-mark clock

1. The stored hour is 7. The clock has 4 marks.

2. 7 % 4 is 3, because 4 * 1 + 3 = 7. The mark is 3.

3. 7 / 4 is 1, the number of full turns. Printing the quotient answers a different question than printing the remainder.

Problem 1. What are 11 / 4 and 11 % 4 when both operands are ints?

Answer. Integer division gives 2. The remainder is 3, because 4 * 2 + 3 = 11.

Problem 2. You wrote if (count = 0) to test whether count is zero. What does that condition do?

Answer. It stores 0 into count. The condition is false, because 0 is the false value. The previous count is gone. A test uses == and does not store.

Lecture 3 · 90 minutes

One branch, then a counted loop

A branch runs one block. if (n > 0) runs the next block only when the test is true, and else runs when it is false. With n equal to 3, you take the true block once. With n equal to 0, you take the else once. Nested tests are still a single path. Draw the two paths before you add a third test, or one value will match none of the blocks you wrote.

A loop repeats a block while a test stays true. for (i = 1; i <= 4; i = i + 1) runs the body four times, with i holding 1, then 2, then 3, then 4. After the loop, i is 5, because the test fails at 5. A while loop is the same machine with the test written at the top. If you forget the update, the test never becomes false and the program does not finish.

A loop that runs four times.

1. i starts at 1

2. Body runs, then i grows

3. Stop when i is 5

4. The body ran 4 times

Add the integers from 1 through 4

1. Set int s = 0 and int i = 1.

2. While i <= 4, set s = s + i and then i = i + 1. The partial sums are 1, then 3, then 6, then 10.

3. The test fails when i is 5. s holds 10. That 10 is the result of the four additions, not a fifth pass.

Problem 1. A for loop sets i to 1, continues while i <= 4, and adds 1 to i each time. How many times does the body run, and what is i after the loop?

Answer. The body runs 4 times, for i equal to 1, 2, 3, and 4. After the loop i is 5, the first value that fails the test.

Problem 2. You want the sum of 1 through 4. The loop adds i, but i stays 1. What happens?

Answer. The test stays true, so the loop does not end. A finished loop adds 1 to i on each pass and stops with the sum 10.

Lecture 4 · 90 minutes

Functions, copies, and a contract

A function is a named block with inputs and one result. The heading int add(int a, int b) says both inputs are int and the result is int. The body returns a + b. A call add(3, 5) copies 3 into a and 5 into b, then returns 8. The copy is the mechanism. If you set a = 0 inside add before the return, you change that copy. The caller's variable is a different slot.

Above the function, you write three short lines: what each argument means, what you return, and what you refuse to do. For add, two ints go in and their sum comes out, with no other effect. A reader should not have to run the body to learn that. If the function also prints, the note must say so. Printing and returning a sum are two jobs. Split them so a test can check the returned 8 without reading the screen.

A call copies, then returns.

Caller writes add(3, 5) -> Copies land in a and b -> Body returns a + b -> Caller receives 8

A function that sums 1 through 4

1. int sum_through_4(void) sets s to 0, then loops i from 1 to 4, adding i into s on each pass.

2. The return value is 10. The names s and i disappear when the function returns.

3. main calls sum_through_4 once and stores 10. It does not repeat the loop itself.

Problem 1. add(4, 6) copies into a and b. The body sets a = 0 and then returns a + b. What does the caller receive, and does the caller's variable change?

Answer. The caller receives 6, because the return uses the copy: 0 + 6. The caller's variable is a separate slot and is not changed.

Problem 2. Why does the three-line note above add have to mention printing if the body prints the sum?

Answer. The contract is inputs, the return, and the effects. Printing is an effect. A caller who only expects the returned 8 will not know the function also writes to the screen unless the note says so.

Lecture 5 · 90 minutes

Arrays, bounds, and the zero byte

An array is one row of slots of a single type. int a[4] gives you four ints, at indexes 0, 1, 2, and 3. The last legal index is 3, one less than the length. a[0] = 10 stores 10 in the first slot. Reading a[4] steps past the row. C does not check that read for you. You pass the length 4 beside the array, because the array value does not store how many slots it has.

A C string is a char array with a zero byte after the last character. The four letters in the word code occupy indexes 0 through 3, and index 4 holds 0. strlen walks until that zero and returns 4. It does not count the zero byte. If you omit the zero, the walk continues into whatever bytes follow. When you copy, leave room for that extra byte, so those four letters need 5 chars.

Four slots, indexes from 0.

Index 0 is the first slot

Index 1 is the second

Index 2 is the third

Index 3 is the last legal

Sum four slots without stepping past them

1. int a[4] holds 1, 2, 3, and 4. Set s to 0 and i to 0.

2. While i < 4, add a[i] into s and add 1 to i. You touch indexes 0, 1, 2, and 3.

3. s is 10. The test i <= 4 would also read index 4, which is outside the four slots.

Problem 1. An array has length 4. Which indexes may you read, and what is index 4?

Answer. You may read 0, 1, 2, and 3. Index 4 is one past the end. It is not a fifth element, and C will not stop you from naming it.

Problem 2. How many chars do you allocate for the word code as a C string, and what does strlen return?

Answer. Allocate 5 chars: the four letters plus a zero byte at index 4. strlen returns 4, because it stops at the zero and does not count it.

Lecture 6 · 90 minutes

Pointers name a slot by its address

A pointer stores an address, not the int sitting at that address. If n is an int, int *p = &n stores the address of n in p. The expression *p reads or writes that int. After *p = 9, n itself is 9. The pointer is a separate slot. When you print p, you see an address. When you print *p, you see 9. Mixing those two prints is the usual bug, and the compiler often accepts both.

When you pass an array, you pass the address of element 0, not a copy of the ints. The parameter int *a holds that address. A write to a[2] changes the caller's slot, because both names refer to the same row of 4 ints. You still pass the length as a second argument. Pointer arithmetic steps by elements: a + 1 is the next int, 4 bytes later. a + 4 is just past the last element.

The pointer and the int.

The pointer p

Holds an address

Printing p shows that address

The int n

Holds 9 after the write

Printing *p shows 9

Write the third slot through a pointer

1. int a[4] starts as 0, 0, 0, 0. int *p = a stores the address of a[0].

2. *(p + 2) = 7 writes 7 into a[2]. The notation p[2] = 7 is the same write.

3. The row is now 0, 0, 7, 0. The caller sees 7 because p holds the row's address, not a copy of four ints.

Problem 1. n is 4. You set int *p = &n and then *p = 9. What is n, and what do you see if you print p rather than *p?

Answer. n is 9, because *p writes the int at that address. Printing p shows the address, not 9. Printing *p shows 9.

Problem 2. A function receives int *a and the length 4, then sets a[0] = 5. Why does the caller see 5 in the first slot?

Answer. The argument is the address of element 0, not a copy of the four ints. a[0] and the caller's first slot are the same memory.

Lecture 7 · 90 minutes

A struct, then one heap block

A struct groups named fields in one object. struct Point { int x; int y; } stores two ints side by side. After you set

p.x = 3 and p.y = 4, both numbers live in p. Pass struct Point by value and the function gets a copy of those two ints. Pass struct Point *q and an assignment through q->x changes the caller's field. Use the pointer when that one object should be updated in place.

malloc(sizeof(struct Point)) asks the heap for one block large enough for 2 ints and returns its address. You test that pointer against null before you write x and y. free takes the same address and returns the block. Call free once. Afterward the pointer still holds the old address, so you set it to null, or a later free will hand that address out a second time. A stack Point dies on return. This heap Point does not.

One heap block, then free.

1. malloc one struct Point

2. Reject a null address

3. Write the two fields

4. free that address once

A heap Point at x = 3, y = 4

1. struct Point *q = malloc(sizeof(struct Point)). Suppose the returned address is not null.

2. Set q->x = 3 and q->y = 4. Both fields sit in that one block.

3. free(q), then set q to null. The block is returned once. You do not pass the old address to free again.

Problem 1. You pass a Point by value into a function that sets x to 8. The caller had x equal to 3. What does the caller see after the return?

Answer. The caller still sees 3. The function received a copy of both fields, and the assignment changed only that copy.

Problem 2. You malloc one Point, free it, then free that same address again. What went wrong?

Answer. The block was returned on the first free. The second free hands the same address out again. After the first free, set the pointer to null so you do not free that address a second time.

Lecture 8 · 90 minutes

A finished program you can defend

You can now write a small C program and name what each piece does. A loop from 1 to 4, a function that returns a sum, an array of length 4, a pointer to its first slot, and one struct on the heap are enough for a utility that reads four ints and returns one result. You also know the note above the function: inputs, the return, and the effects. Pass every length. Test malloc. free each block once.

The failure you look for is a contract that disagrees with the body. If the comment says you do not allocate, but the body calls malloc for 4 ints, the next reader will leak those ints. Match the comment to the code before you stop. Run the loop by hand on the four values and write the sum. If the hand sum and the return disagree, the bug is the update or the bound. Fix the function, then the comment.

What a finished utility contains.

A note on inputs and return

A loop with a tested bound

An array passed with its length

One malloc paired with one free

Sum four ints that live on the heap

1. int *a = malloc(4 * sizeof(int)). If a is null, stop and allocate nothing else.

2. Store 1, 2, 3, and 4. int total(int *a, int n) with n equal to 4 adds a[0] through a[3] and returns 10.

3. free(a) and set a to null. The note says the caller owns the block and frees it once. The function total itself does not allocate.

Problem 1. The note says total does not allocate and sums 4 ints. The body mallocs 4 ints, loops while i <= 4, adds a[i], frees once, and returns. The slots hold 1, 2, 3, and 4. What is wrong, and what should a correct sum be?

Answer. Two faults, from the contract and from the bound. The note says no allocation, but malloc ran. The test i <= 4 reads index 4, past the last legal index 3. The sum of the four ints is 10, using indexes 0, 1, 2, and 3 only. If the malloc stays, free it once and fix the note.

Problem 2. You need a function to set a Point's y to 4 while x stays 3, and the caller must see both fields. Do you pass the struct by value or by pointer?

Answer. Pass struct Point * and assign through the pointer. After the call, x is 3 and y is 4. A by-value parameter would change a copy, and the caller would keep the old y.